Git

03 - Workflow

Introduction

- Vous savez désormais :
  - Initialiser un projet Git localement
  - Créer un dépôt distant sur GitHub
  - Associer un dépôt local à un dépôt distant
  - Utiliser des commandes élémentaires, ex. :
    - `git init`
    - `git clone`
    - `git add` / `git stage`
    - `git commit`
    - `git push`
- **Questions :**
  - Quelle est la **structure d'un historique** de versions ?
  - Comment organise-t-on le **travail à plusieurs** sur un même projet ?

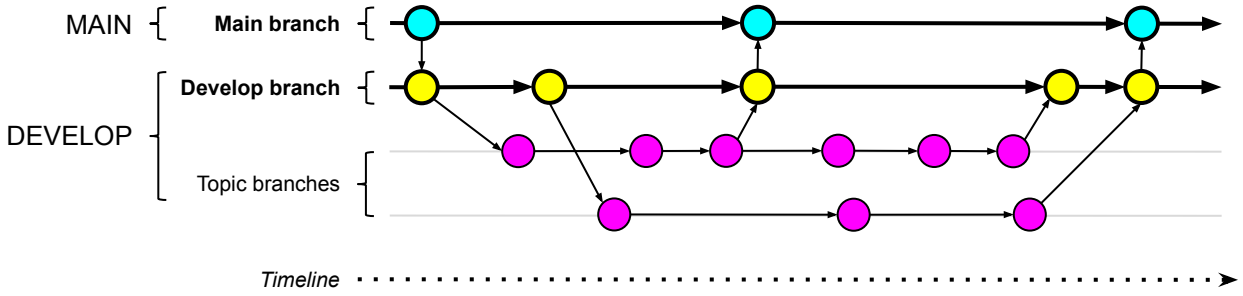
Notion de *workflow*

- Trad. littérale : ***« Flux de travail »***
- Définit la façon dont un historique de versions est construit :
  - notion d'**arbre de versions** : *versions tree*
  - notion de **branches** : *branches*
  - adoption de **conventions**, ex. :
    - une fonctionnalité = un sous-embranchement
    - fréquence des *commits*
    - nommage des *commits*
- Permet de conserver un **historique *« propre »*** et lisible
- Permet une bonne **coordination de l'équipe**
- Permet la **mise en production** sans risque d'une version stable

Notion de *workflow*

Exemple de ***workflow* simple** :

- Une branche principale : *main*
- Une branche dédiée à l'avancement du développement du projet : *develop*
- n branches secondaires : *topic branches*
- Repère temporel (avancement du projet) : *timeline*



Workflow simple : Explications

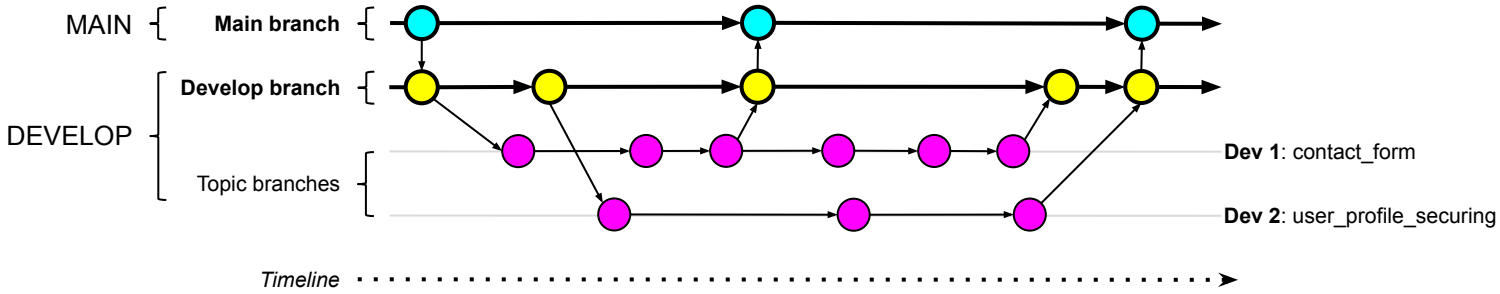
- Branche ***main*** : **état stable** d'une version pour la **mise en production** (*release*)
- Branche ***develop*** : **état d'avancement** du développement du projet
- Branches secondaires ***topic*** (ou *feature*) :
  - Branches de travail quotidienne des développeurs
  - Branches dérivées de la branche *develop*
  - Branches fusionnées dans la branche *develop* <u>une fois le travail terminé</u>

Workflow simple : Explications

- Notion de ***topic*** (ou *feature*) :
  - Un *topic* (thème) constitue un point spécifique **à implémenter**
  - Fréquemment, *topic* = *feature* (c.-à-d. **une fonctionnalité**)
  - Ex. : *« Formulaire de contact »*, *« Sécurisation du profil utilisateur »*, etc.
  - Nom de branche (`snake_case`) : `contact_form`, `user_profile_securing` , etc.
- Notion de branche **dérivée** :
  - Contient l'état actuel *« figé »* de la branche dont elle est dérivée
  - Évolue en parallèle de cette dernière
  - Commande : `git checkout -b`

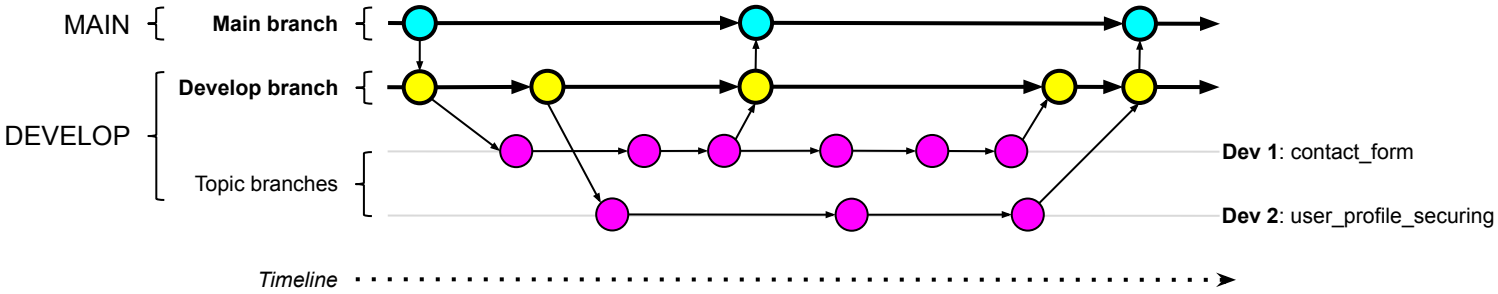
Workflow simple : Explications

- **Dev 1** a dérivé la branche `contact_form` à partir de la branche `develop`
- **Dev 2** a dérivé la branche `user_profile_securing` à partir de la branche `develop`
- **Dev 1** et **Dev 2** travaillent en parallèle (c.-à-d. de manière **non linéaire**)



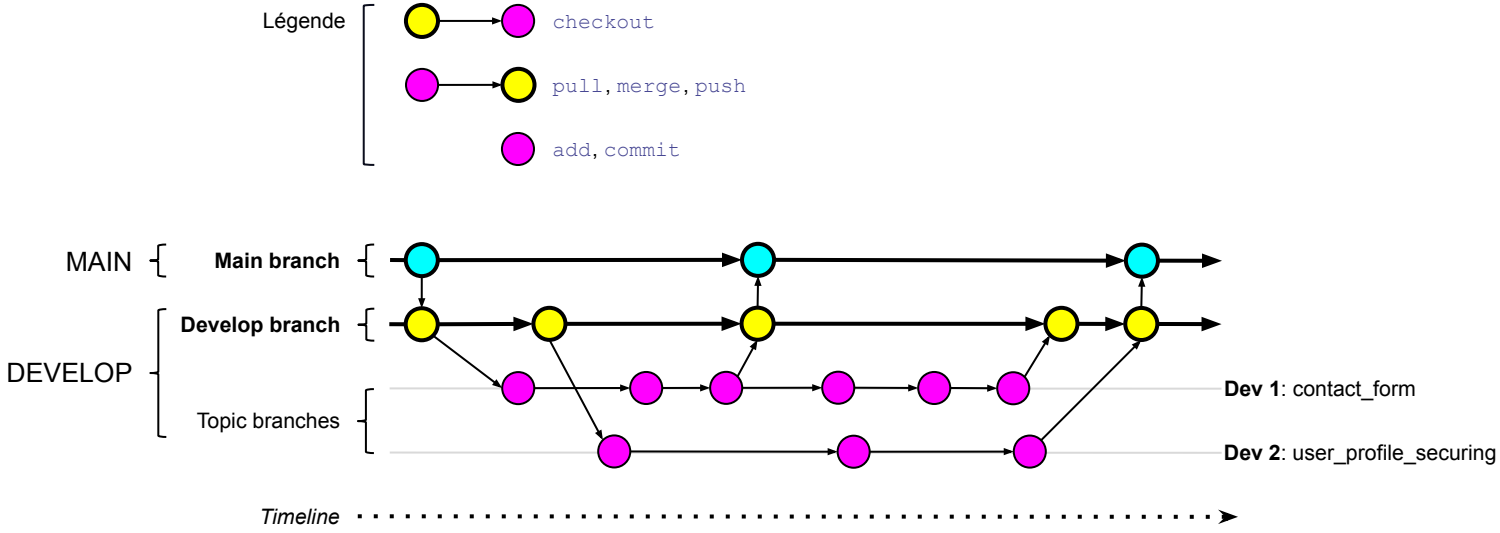
Workflow simple : Explications

- **Dev 1** progresse sur sa branche `contact_form` :
  - il **enregistre** régulièrement l'état d'avancement (`add`, `commit`)
  - il **fusionne** certains états d'avancement dans la branche develop (`pull`, `merge`, `push`)
  - une fois le travail complètement terminé, il peut entamer le développement d'une nouvelle fonctionnalité (donc, une **nouvelle branche**)

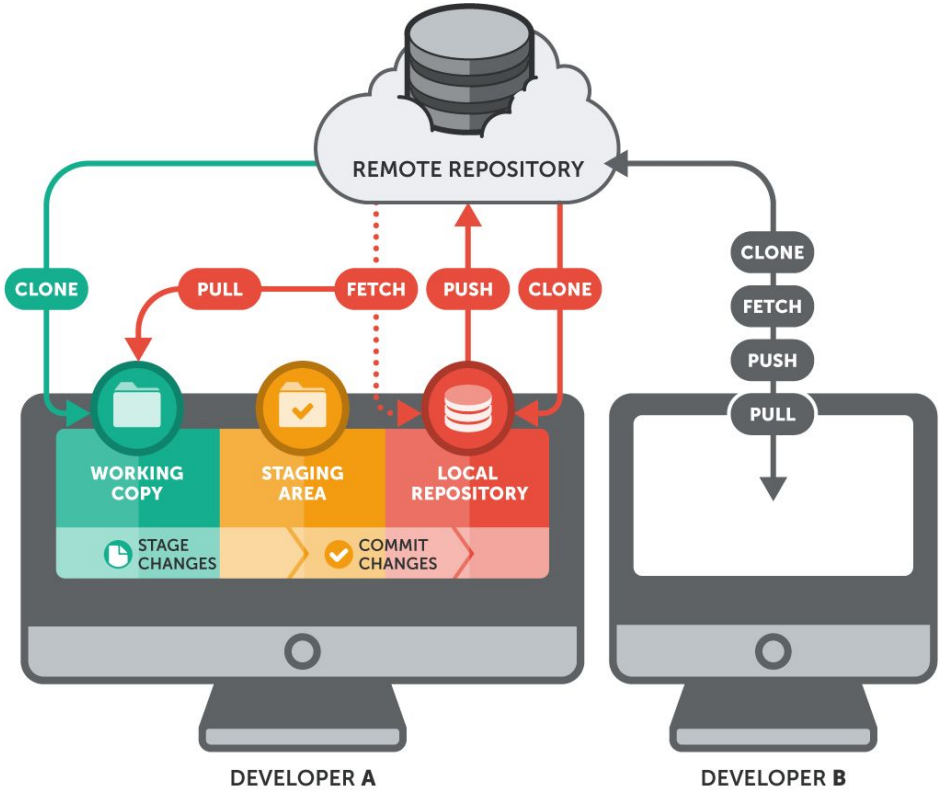


Workflow simple : Explications

- Progression de **Dev 1** sur la branche `contact_form`



Workflow simple : Synthèse



Conclusion

- Progression similaire de **Dev 2** sur la branche `user_profile_securing`
- Détail de la branche ***main*** : dans une mission ultérieure
- *« Routine »* quotidienne :
  - Répétition d'une séquence de commandes Git
  - Conventions (nommage des branches, messages associés aux *commits*)
- **Document de synthèse** (fourni en annexe) : `Git-Synthèse.pdf`
  - Contient l'essentiel de l'intégralité de la quête
  - À conserver en guise d'**aide-mémoire**

> > > MISSION START ! < < <